Penetration Test Report

Executive Summary

This report outlines vulnerabilities discovered during testing.

Findings

Vulnerability Report

Title: Reflected Cross-Site Scripting (XSS) in Search Parameter

Vulnerability ID: XSS-2023-001

Description:

A reflected Cross-Site Scripting (XSS) vulnerability was identified in the `/search` endpoint, which accepts user-controlled input via the `q` parameter. An attacker can inject malicious JavaScript code by passing crafted input, allowing for potential exploitation and compromise of user authentication sessions.

Impact:

Successful exploitation of this vulnerability can lead to:

1. **Session Hijacking**: An attacker can steal user session cookies, gaining unauthorized access to sensitive data and resources.
2. **Arbitrary Code Execution**: Malicious JavaScript code can be executed in the context of the user session, leading to further exploitation and potential remote code execution.
3. **Data Exfiltration**: Sensitive data can be collected by the attacker through malicious scripts, posing a significant risk to user confidentiality.

Steps to Reproduce:

1. Perform a GET request to `http://target.com/search?q=<script>alert('XSS')</script>`.
2. Observe the response and note the presence of the injected script.
3. Confirm that the script executes and the alert is displayed in the user's browser.

Remediation:

1. **Input Validation**: Implement robust input validation and sanitization for the `q` parameter to prevent malicious characters from being injected.
2. **Output Encoding**: Encode user input using HTML encoding to prevent script injection.

3. **Content Security Policy (CSP)**: Implement a CSP to define allowed and disallowed sources of scripts, ensuring that malicious JavaScript is not executed.

Severity: Critical

CVSS Score: 10.0

CVSS Vector: AV:N/AC:L/PR:N

Vulnerability Report

Title: Vulnerable Login Form SQL Injection **Vulnerability ID:** PEN-2023-01

Description: The login form at '/login' endpoint is vulnerable to SQL injection attacks via the POST method. By manipulating user input, an attacker can inject malicious SQL code to extract sensitive data, modify existing data, or escalate privileges.

Impact: An unauthorized attacker can gain access to sensitive user credentials, inject malware, or potentially gain administrative privileges, compromising the confidentiality, integrity, and availability of the system. Real-world consequences include data breaches, unauthorized access, and reputational damage.

Steps to Reproduce:

1. Send a POST request to the '/login' endpoint with the following malicious payload: `username=' OR 1=1 --`
2. Observe the system's response and verify that the malicious payload has been successfully injected into the SQL query.

Remediation:

1. **Input Validation and Sanitization**: Implement input validation to restrict user input to alphanumeric characters only. Sanitize user input by removing or escaping any special characters.
2. **Parametrized Queries**: Update the login form to use parametrized queries to prevent SQL injection attacks. Utilize a prepared statement or a parameterized query library (e.g., Python's `sqlalchemy`).
3. **Least Privilege**: Ensure that the database user account running the login form has the minimum required privileges to prevent an attacker from escalating privileges in case of an attack.
4. **Regular Security Audits**: Schedule regular security audits to identify and address potential vulnerabilities.

Severity: Critical (CVSS Score: 10.0)

CVE: (Awaiting assignment)

Recommendations: Implement the above remediation steps to prevent further SQL injection attacks.

Security Vulnerability Report

Title: User Data Exfiltration via Insecure Direct Object Reference (IDOR)
Vulnerability ID: V0010-CRITICAL

Description: The `/api/user` endpoint is vulnerable to Insecure Direct Object Reference (IDOR) attacks. This allows unauthorized users to access sensitive user data by manipulating the URL parameter. Specifically, the endpoint retrieves user information based on the provided `id` parameter without adequate authorization checks.

Technical Details: - Endpoint: `/api/user` - HTTP Method: `GET` - Affected Resource: User data

Impact: An attacker can exploit this vulnerability to access sensitive information of arbitrary users, potentially leading to identity theft, data breaches, or unauthorized account takeover.

Steps to Reproduce:

1. Access the `/api/user` endpoint using a tool like `curl` or a web browser.
2. Manipulate the `id` parameter to retrieve user data for an arbitrary ID (e.g., `127.0.0.1/api/user?id=1234567890`).
3. Verify that the application does not authenticate or authorize the request.

Remediation: To mitigate this vulnerability:

1. Implement proper authentication and authorization checks for all API endpoints.
2. Validate and sanitize user input to ensure the `id` parameter is not manipulated.
3. Update existing `/api/user` endpoint to require valid user sessions or authentication tokens.
4. Consider implementing a secure IDOR protection mechanism, such as using a separate authorization service.

Severity: Critical **CVSS Score:** 9.1 **CVSS Vector:** AV:N/AC:L/PR:L

Recommendations:

- Immediately deploy patches or fixes to affected systems.
- Review and update all insecure endpoints to follow a secure design pattern.
- Conduct regular security testing and code reviews to prevent similar vulnerabilities.